Vector

Copy page

These functions perform vector operations used in numerical computation, machine learning, and data analysis. They include distance measurements, similarity coefficients, and other operations on vectors.

Function	Description
`vector::add()`	Performs element-wise addition of two vectors
`vector::angle()`	Computes the angle between two vectors
`vector::cross()`	Computes the cross product of two vectors
`vector::divide()`	Performs element-wise division between two vectors
`vector::dot()`	Computes the dot product of two vectors
`vector::magnitude()`	Computes the magnitude (or length) of a vector
`vector::multiply()`	Performs element-wise multiplication of two vectors
`vector::normalize()`	Computes the normalisation of a vector
`vector::project()`	Computes the projection of one vector onto another
`vector::scale()`	Multiplies each item in a vector
`vector::sum()`	Sums vectors element-wise, as a scalar or as a grouped aggregate
`vector::subtract()`	Performs element-wise subtraction between two vectors
`vector::distance::chebyshev()`	Computes the Chebyshev distance
`vector::distance::euclidean()`	Computes the Euclidean distance between two vectors

Skip to content

Function	Description
`vector::distance::hamming()`	Computes the Hamming distance between two vectors
`vector::distance::knn()`	Returns the distance computed during the query
`vector::distance::manhattan()`	Computes the Manhattan distance between two vectors
`vector::distance::mahalanobis()`	Computes the Mahalanobis distance between two vectors given a covariance matrix
`vector::distance::minkowski()`	Computes the Minkowski distance between two vectors
`vector::similarity::cosine()`	Computes the Cosine similarity between two vectors
`vector::similarity::jaccard()`	Computes the Jaccard similarity between two vectors
`vector::similarity::pearson()`	Computes the Pearson correlation coefficient between two vectors
`vector::similarity::spearman()`	Computes the Spearman rank correlation coefficient between two vectors

`vector::add`

The `vector::add` function performs element-wise addition of two vectors, where each element in the first vector is added to the corresponding element in the second vector.

```
API DEFINITION

vector::add(array, $other: array) -> array
```

The following example shows this function, and its output:

```
vector::add([1, 2, 3], [1, 2, 3]);
```

```
Output
```

Skip to content

vector::angle

The `vector::angle` function computes the angle between two vectors, providing a measure of the orientation difference between them.

API DEFINITION

```
vector::angle(array, $other: array) -> number
```

Both vectors must have the same dimension and a non-zero magnitude. A zero-magnitude input (including `[]`) returns an error.

The following example shows this function, and its output:

```
vector::angle([5, 10, 15], [10, 5, 20]);
```

Output

```
0.36774908225917935f
```

vector::cross

The `vector::cross` function computes the cross product of two vectors, which results in a vector that is orthogonal (perpendicular) to the plane containing the original vectors.

API DEFINITION

```
vector::cross(array, $other: array) -> array
```

The following example shows this function, and its output:

```
vector::cross([1, 2, 3], [4, 5, 6]);
```

```
[-3, 6, -3]
```

Skip to content

vector::divide

The `vector::divide` function performs element-wise division between two vectors, where each element in the first vector is divided by the corresponding element in the second vector.

API DEFINITION

```
vector::divide(array, $other: array) -> array
```

The divisor vector must not contain a zero. Division by zero returns an error instead of `NaN`.

The following example shows this function, and its output:

```
vector::divide([4, 6], [2, 3]);
```

Output

```
[2, 2]
```

vector::dot

The `vector::dot` function computes the dot product of two vectors, which is the sum of the products of the corresponding entries of the two sequences of numbers.

API DEFINITION

```
vector::dot(array, $other: array) -> number
```

The following example shows this function, and its output:

```
vector::dot([1, 2, 3], [1, 2, 3]);
```

Output

```
14
```

Skip to content

vector::magnitude

The `vector::magnitude` function computes the magnitude (or length) of a vector, providing a measure of the size of the vector in multi-dimensional space.

API DEFINITION

```
vector::magnitude(array) -> number
```

An empty vector returns `0`.

The following example shows this function, and its output:

```
vector::magnitude([ 1, 2, 3, 3, 3, 4, 5 ]);
```

Output

```
8.544003745317753f
```

vector::multiply

The `vector::multiply` function performs element-wise multiplication of two vectors, where each element in the first vector is multiplied by the corresponding element in the second vector.

API DEFINITION

```
vector::multiply(array, $other: array) -> array
```

The following example shows this function, and its output:

```
vector::multiply([1, 2, 3], [1, 2, 3]);
```

Output

```
[1, 4, 9]
```

Skip to content

vector::normalize

The `vector::normalize` function computes the normalisation of a vector, transforming it to a unit vector (a vector of length 1) that maintains the original direction.

API DEFINITION

```
vector::normalize(array) -> array
```

The vector must have a non-zero magnitude. A zero vector (including `[]`) returns an error.

The following example shows this function, and its output:

```
vector::normalize([ 4, 3 ]);
```

Output

```
[0.8f, 0.6f]
```

vector::project

The `vector::project` function computes the projection of one vector onto another, providing a measure of the shadow of one vector on the other. The projection is obtained by multiplying the magnitude of the given vectors with the cosecant of the angle between the two vectors.

API DEFINITION

```
vector::project(array, $other: array) -> array
```

The second vector must have a non-zero magnitude. Projecting onto a zero vector (including `[]`) returns an error.

The following example shows this function, and its output:

```
vector::project([1, 2, 3], [4, 5, 6]);
```

Skip to content

```
[1.6623376623376624f, 2.077922077922078f, 2.4935064935064934f]
```

vector::scale

The `vector::scale` function multiplies each item in a vector by a number.

```
API DEFINITION

vector::scale(array, $other: number) -> array
```

The following example shows this function, and its output:

```
vector::scale([3, 1, 5, -3, 7, 2], 5);
```

```
Output

[15,    5, 25, -15, 35, 10]
```

vector::sum

AVAILABLE SINCE: V3.3.0

The `vector::sum` function adds vectors element-wise. It is available in two forms, matching other aggregates such as `math::sum`:

- **Scalar** - sum a collection of vectors you already hold as `array<array<number>>`
- **Aggregate** - fold a vector-valued expression across the rows of a `GROUP BY` (or `GROUP ALL`)

```
API DEFINITION

vector::sum(array<array<number>>) -> array | none
```

Skip to content

tion returns `NONE` (there is no dimension to produce a zero vector for). Two um to `[]`. Vectors in the collection must share the same dimension; a mismatch returns an error in the scalar form.

The following example shows the scalar form, and its output:

```
vector::sum([[1, 2, 3], [4, 5, 6]]);
//- [5, 7, 9]

vector::sum([[1, 2], [3, 4], [5, 6]]);
//- [9, 12]

vector::sum([[1.5, 2.5], [1, 1]]);
//- [2.5f, 3.5f]

vector::sum([[], []]);
//- []

vector::sum([]);
//- NONE

[[1, 2], [3, 4]].vector_sum();
//- [4, 6]
```

Aggregate form

As an aggregate, `vector::sum` keeps one running vector per group, so state stays O(dimension) rather than materialising every embedding with `array::group` then folding.

On the streaming path:

- Rows whose vector is `NONE` or `NULL` are skipped (same idea as a missing field for `math::sum`)
- A group that never sees a vector yields `NONE`
- A group whose vectors disagree on dimension yields `NULL`, not a partial sum

Together with `vector::scale` and `math::sum`, a weighted centroid is one query:

```
CREATE engagement:1 SET user = 'alice', weight = 2, embedding = [1, 0,
CREATE engagement:2 SET user = 'alice', weight = 3, embedding = [0, 1,
CREATE engagement:3 SET user = 'bob', weight = 1, embedding = [0, 0, 4
```

Skip to content

```
    vector::scale(
```

```
        vector::sum(vector::scale(embedding, weight)),
        1.0 / math::sum(weight)
    ) AS interest
FROM engagement
GROUP BY user
ORDER BY user;

-- alice → [0.4f, 0.6000000000000001f, 0f]
-- bob   → [0f, 0f, 4f]

SELECT user, vector::sum(embedding) AS total
FROM engagement
GROUP BY user
ORDER BY user;

-- alice → [1, 1, 0]
-- bob   → [0, 0, 4]
```

> **Important**
>
> Write the reciprocal as a float (`1.0 / math::sum(weight)`). With integer weights, `1 / math::sum(weight)` is integer division and truncates to `0`, which scales the weighted total away entirely. Float weights promote the reciprocal on their own.

`vector::subtract`

The `vector::subtract` function performs element-wise subtraction between two vectors, where each element in the second vector is subtracted from the corresponding element in the first vector.

```
API DEFINITION
```

Skip to content

```
tract(array, $other: array) -> array
```

The following example shows this function, and its output:

```
vector::subtract([4, 5, 6], [3, 2, 1]);
```

Output

```
[1, 3, 5]
```

vector::distance::chebyshev

The `vector::distance::chebyshev` function computes the Chebyshev distance (also known as maximum value distance) between two vectors, which is the greatest of their differences along any coordinate dimension.

API DEFINITION

```
vector::distance::chebyshev(array, $other: array) -> number
```

Two empty vectors return `0`.

The following example shows this function, and its output:

```
vector::distance::chebyshev([2, 4, 5, 3, 8, 2], [3, 1, 5, -3, 7, 2]);
```

Output

```
6f
```

vector::distance::euclidean

The `vector::distance::euclidean` function computes the Euclidean distance between two vectors, providing a measure of the straight-line distance between two points in a multi-dimensional space.

Skip to content

```
vector::distance::euclidean(array, $other: array) -> number
```

The following example shows this function, and its output:

```
vector::distance::euclidean([10, 50, 200], [400, 100, 20]);
```

```
Output

432.43496620879307f
```

vector::distance::hamming

The `vector::distance::hamming` function computes the Hamming distance between two vectors, measuring the minimum number of substitutions required to change one vector into the other, useful for comparing strings or codes.

```
API DEFINITION

vector::distance::hamming(array, $other: array) -> number
```

The following example shows this function, and its output:

```
vector::distance::hamming([1, 2, 2], [1, 2, 3]);
```

```
Output

1
```

vector::distance::knn

The `vector::distance::knn` function returns the distance computed during the query by the Knn operator (avoiding recomputation).

Skip to content

```
vector::distance::knn() -> number
```

The following example shows this function, and its output, when used in a `SELECT` statement:

```
CREATE pts:1 SET point = [1,2,3,4];
CREATE pts:2 SET point = [4,5,6,7];
CREATE pts:3 SET point = [8,9,10,11];
SELECT id, vector::distance::knn() AS dist FROM pts
  WHERE point <|2,EUCLIDEAN|> [2,3,4,5];
```

Output

```
[
    {
        id: pts:1,
        dist: 2f
    },
    {
        id: pts:2,
        dist: 4f
    }
]
```

vector::distance::manhattan

The `vector::distance::manhattan` function computes the Manhattan distance (also known as the L1 norm or Taxicab geometry) between two vectors, which is the sum of the absolute differences of their corresponding elements.

API DEFINITION

```
vector::distance::manhattan(array, $other: array) -> number
```

The following example shows this function, and its output:

Skip to content

```
vector::distance::manhattan([10, 20, 15, 10, 5], [12, 24, 18, 8, 7]);
```

Output

```
13
```

vector::distance::mahalanobis

AVAILABLE SINCE: V3.3.0

The `vector::distance::mahalanobis` function computes the Mahalanobis distance between two vectors given a covariance matrix. Unlike Euclidean distance, it scales differences by the inverse of the covariance, so correlated dimensions are not double counted.

API DEFINITION

```
vector::distance::mahalanobis(array, $other: array, $covariance: array
```

The two vectors must share the same non-zero dimension. The covariance argument must be a square matrix of that dimension and must be symmetric positive-definite (validated via Cholesky decomposition). When the covariance is the identity matrix, the result matches `vector::distance::euclidean`.

The following example shows this function, and its output:

```
-- An identity covariance matrix gives the same result as euclidean
vector::distance::mahalanobis([1, 2], [3, 4], [[1, 0], [0, 1]]);
//- 2.8284271247461903f

vector::distance::mahalanobis([1, 2], [3, 4], [[2, 1], [1, 2]]);
//- 1.6329931618554520f

vector::distance::mahalanobis([1, 2], [1, 2], [[2, 1], [1, 2]]);
//- 0f
```

A non-square matrix, a matrix that is not symmetric positive-definite, empty vectors, or a dimension mismatch returns an error.

Skip to content

vector::distance::minkowski

The `vector::distance::minkowski` function computes the Minkowski distance between two vectors, a generalization of other distance metrics such as Euclidean and Manhattan when parameterised with different values of p.

```
API DEFINITION

vector::distance::minkowski(array, $other: array, $p_value: number) ->
```

The following example shows this function, and its output:

```
vector::distance::minkowski([10, 20, 15, 10, 5], [12, 24, 18, 8, 7], 3
```

```
Output

4.862944131094279f
```

vector::similarity::cosine

The `vector::similarity::cosine` function computes the Cosine similarity between two vectors, indicating the cosine of the angle between them, which is a measure of how closely two vectors are oriented to each other.

```
API DEFINITION

vector::similarity::cosine(array, $other: array) -> number
```

The following example shows this function, and its output:

```
vector::similarity::cosine([10, 50, 200], [400, 100, 20]);
```

```
Output

0.15258215962441316f
```

Skip to content

vector::similarity::jaccard

The `vector::similarity::jaccard` function computes the Jaccard similarity between two vectors, treating each vector as a set of numbers (intersection size divided by union size). Duplicate values in a vector count once.

```
API DEFINITION

vector::similarity::jaccard(array, $other: array) -> number
```

Two empty vectors return `1` (both sets are empty).

The following example shows this function, and its output:

```
vector::similarity::jaccard([0,1,2,5,6], [0,2,3,4,5,7,9]);
//- 0.3333333333333333f

-- The sets are {1,2} and {2}: one item in the intersection, two in th
vector::similarity::jaccard([1, 2], [2, 2]);
//- 0.5f
```

vector::similarity::pearson

The `vector::similarity::pearson` function computes the Pearson correlation coefficient between two vectors, reflecting the degree of linear relationship between them.

```
API DEFINITION

vector::similarity::pearson(array, array) -> number
```

Both vectors must have the same dimension of at least 2, and neither may have zero variance.

The following example shows this function, and its output:

```
vector::similarity::pearson([1,2,3], [1,5,7]);
```

Skip to content

```
0.9819805060619659f
```

vector::similarity::spearman

AVAILABLE SINCE: V3.3.0

The `vector::similarity::spearman` function computes the Spearman rank correlation between two vectors: each vector is converted to average ranks (ties share a rank), then Pearson correlation is applied to those ranks.

API DEFINITION

```
vector::similarity::spearman(array, $other: array) -> number
```

Both vectors must have the same dimension of at least 2, and neither may have zero variance after ranking (a constant vector returns an error).

The following example shows this function, and its output:

```
vector::similarity::spearman([1, 2, 3], [1, 10, 100]);
//- 1f

vector::similarity::spearman([1, 2, 3], [3, 2, 1]);
//- -1f

-- Ties use average ranks
vector::similarity::spearman([1, 2, 2, 3], [1, 2, 3, 4]);
//- 0.9486832980505138f
```

Skip to content

The context and memory layer for AI agents

Graph, vector, document, and relational in one engine.
Agent Memory that connects and retrieves context wherever your data lives.

DOCUMENTATION

- Overview
- Agent Memory
- SurrealQL
- SDKs
- Cloud
- Labs

RESOURCES

- University
- Blog
- Case studies
- Releases
- Roadmap
- Events and webinars

COMPANY

- Contact us
- Careers
- Open source
- Brand assets
- Status
- Contributing

LEGAL

- Overview
- Terms
- Privacy
- Trust and security
- Compliance
- Licence FAQs

COMMUNITY

Discord | GitHub | GitHub Discussions | X | YouTube | LinkedIn | Reddit | Instagram | Stack Overflow | Medium | Dev

© SurrealDB 2026